



Calidad de Software y Estrategia de Validación

18 de julio de 2026

Tabla de contenido

1. Introducción	4
2. Objetivos de calidad	5
3. Atributos de calidad priorizados.....	6
4. Justificación de prioridades	7
5. Escenarios de calidad	8
6. Tácticas arquitectónicas	11
7. Relación entre atributos y decisiones arquitectónicas.....	13
8. Estrategia de pruebas	14
9. Pruebas de integración	15
10. Pruebas E2E.....	17
11. Pruebas E2E	19
12. Pruebas de rendimiento	20
13. Pruebas de seguridad.....	22
14. Pruebas de usabilidad	25
15. Análisis estático	27
16. Cobertura de código	30
17. Métricas de calidad	31
18. Métricas de calidad	33
19. Resultados obtenidos.....	34
20. Riesgos de calidad.....	36
21. Evidencias	37
22. Conclusiones.....	39

1. Introducción

ECIexpress es una plataforma de pedidos universitarios desarrollada bajo una arquitectura basada en microservicios, diseñada para gestionar de manera integral el proceso de compra dentro del entorno universitario. La solución está compuesta por ocho microservicios independientes que se comunican a través de un API Gateway, permitiendo administrar funcionalidades relacionadas con autenticación, catálogo de productos, gestión de órdenes, pagos, notificaciones, logística y generación de reportes. La infraestructura del sistema se ejecuta mediante Docker Compose e integra tecnologías como PostgreSQL, Redis y RabbitMQ para el almacenamiento de datos, el manejo de sesiones y la comunicación asíncrona entre servicios.

El presente documento describe la estrategia de calidad de software implementada para validar el comportamiento funcional y no funcional de la plataforma. En él se presentan los atributos de calidad priorizados, los escenarios de evaluación, las tácticas arquitectónicas empleadas para cumplir dichos atributos y la estrategia de pruebas utilizada durante el proceso de validación.

La evaluación incluyó pruebas funcionales, pruebas de integración, pruebas End-to-End (E2E), validación de contratos mediante DTOs, pruebas de rendimiento bajo carga, análisis de seguridad, análisis estático de código y verificación de cobertura funcional. Asimismo, se documentan los resultados obtenidos, los defectos identificados, los riesgos asociados a la calidad del sistema y las evidencias recopiladas durante la ejecución de las pruebas.

El objetivo principal de este documento es demostrar que la plataforma ECIexpress cumple con los criterios de calidad definidos para su operación, proporcionando una visión integral del estado del sistema, su nivel de confiabilidad y las oportunidades de mejora identificadas durante el proceso de validación.

2.Objetivos de calidad

El proceso de validación de ECIxpress tiene como propósito verificar que la plataforma cumple con los atributos de calidad definidos para una arquitectura basada en microservicios. Para ello, se establecieron los siguientes objetivos:

- Verificar que más del 90 % de los endpoints del sistema respondan correctamente bajo condiciones normales de operación, garantizando el correcto funcionamiento de los servicios expuestos.
- Validar que la plataforma sea capaz de soportar al menos 50 usuarios concurrentes, manteniendo una latencia aceptable y sin presentar degradaciones significativas en el desempeño.
- Identificar vulnerabilidades de seguridad presentes tanto en el código fuente como en las dependencias, la configuración de infraestructura y los componentes utilizados por la solución.
- Detectar posibles secretos o credenciales expuestas dentro del repositorio mediante herramientas especializadas de análisis de seguridad.
- Garantizar la integridad de la información utilizada durante las pruebas, asegurando que todos los datos de prueba creados sean eliminados correctamente al finalizar su ejecución y evitando afectar el entorno de trabajo.
- Documentar los defectos encontrados durante el proceso de validación, registrando su severidad, impacto, causa raíz y posibles alternativas de solución.
- Establecer una línea base de rendimiento que permita comparar futuras versiones del sistema y evaluar la evolución de la calidad del software a lo largo del ciclo de vida del proyecto.

3. Atributos de calidad priorizados

Los atributos de calidad representan las características que determinan el comportamiento esperado del sistema desde una perspectiva funcional y no funcional. En ECIxpress se priorizaron aquellos atributos que tienen un mayor impacto sobre la experiencia del usuario, la continuidad del servicio y la mantenibilidad de la plataforma. Cada atributo fue definido junto con una métrica de evaluación y un objetivo que permitió medir su cumplimiento durante el proceso de validación.

Atributo	Prioridad	Métrica	Objetivo
Funcionalidad	Crítica	Endpoints funcionales / total	> 90 %
Rendimiento	Crítica	Latencia p(95) bajo carga	< 500 ms
Disponibilidad	Alta	Health checks respondiendo	100 % de los servicios activos
Seguridad	Alta	Vulnerabilidades críticas	0 vulnerabilidades críticas
Mantenibilidad	Alta	Deuda técnica (Semgrep)	Menos de 20 hallazgos críticos
Integridad de datos	Alta	Limpieza de datos de prueba	100 % de los datos eliminados
Escalabilidad	Media	Throughput máximo sostenido	> 50 solicitudes por segundo

4. Justificación de prioridades

En una plataforma basada en microservicios como ECIxpress, no todos los atributos de calidad tienen el mismo impacto sobre la operación del sistema. Por esta razón, se estableció un orden de prioridad considerando el riesgo asociado a cada atributo, el impacto sobre el negocio y la experiencia de los usuarios.

Funcionalidad

- La funcionalidad constituye el atributo más importante del sistema, ya que ECIxpress es una plataforma transaccional donde cada endpoint debe cumplir el comportamiento definido por su contrato. Un error en la creación de órdenes, el manejo del catálogo o el procesamiento de pagos afecta directamente la operación del negocio.

Rendimiento

- El sistema está diseñado para soportar picos de demanda durante los horarios de mayor actividad universitaria. Por ello, se estableció como objetivo mantener tiempos de respuesta bajos y validar el comportamiento bajo múltiples usuarios concurrentes.

Disponibilidad

- La arquitectura está compuesta por ocho microservicios que trabajan de forma coordinada. La indisponibilidad de servicios críticos como Gateway, Orders o Products puede impedir el funcionamiento normal de la plataforma.

Seguridad

- ECIxpress gestiona información de usuarios, autenticación y operaciones relacionadas con pagos. Por esta razón, se realizaron análisis de vulnerabilidades y validaciones de seguridad para reducir los riesgos asociados al sistema.

Mantenibilidad

- Al estar distribuido en múltiples microservicios, mantener un código limpio y bien estructurado facilita la evolución, el mantenimiento y la incorporación de nuevas funcionalidades.

5. Escenarios de calidad

Los escenarios de calidad permiten evaluar el comportamiento del sistema frente a situaciones representativas de su operación. Cada escenario define una fuente de estímulo, el evento que ocurre, el entorno donde se presenta, el componente afectado, la respuesta esperada y el criterio utilizado para medir su cumplimiento. Estos escenarios fueron seleccionados con el objetivo de validar los atributos de calidad priorizados para la plataforma ECIexpress.

Consulta de catálogo bajo carga

Elemento	Descripción
Fuente del estímulo	50 usuarios concurrentes.
Estímulo	Solicitudes GET a /products, /categories y /promotions.
Entorno	Plataforma desplegada con ocho microservicios en Docker, PostgreSQL y Redis.
Artefacto	Products Service y API Gateway.
Respuesta esperada	Respuestas HTTP 200 con información completa del catálogo.
Medida de respuesta	Latencia promedio inferior a 100 ms y percentil 95 inferior a 300 ms.

Creación de orden completa

Elemento	Descripción
Fuente del estímulo	Usuario autenticado.
Estímulo	Flujo de creación de orden: Draft → Agregar productos → Cotización → Checkout.
Entorno	Orders Service, Products Service y Financial Service desplegados en Docker.
Artefacto	Orders Service.
Respuesta esperada	La orden se crea correctamente y avanza hasta el estado PENDING_PAYMENT .
Medida de respuesta	Cada operación responde en menos de 2 segundos.

Autenticación de usuario nuevo

Elemento	Descripción
Fuente del estímulo	Usuario no registrado previamente.
Estímulo	Registro mediante Firebase, sincronización del perfil y validación del usuario.
Entorno	Firebase Authentication e Identity Service.
Artefacto	Identity Service.
Respuesta esperada	Creación del perfil y validación exitosa del usuario.
Medida de respuesta	La validación responde con HTTP 200 en menos de un segundo.

Protección de endpoints autenticados

Elemento	Descripción
Fuente del estímulo	Usuario sin credenciales o con un token inválido.
Estímulo	Solicitudes a endpoints protegidos como usuarios, órdenes y billetera.
Entorno	API Gateway e Identity Service.
Artefacto	Mecanismo de autenticación.
Respuesta esperada	El sistema responde con HTTP 401 Unauthorized.
Medida de respuesta	El 100 % de los endpoints protegidos rechazan solicitudes sin autenticación válida.

Recuperación ante fallo de servicio externo

Elemento	Descripción
Fuente del estímulo	Indisponibilidad de la pasarela de pagos Wompi.
Estímulo	Solicitud de recarga utilizando BANCOLOMBIA_TRANSFER.
Entorno	Financial Service conectado a la API de Wompi.
Artefacto	Financial Service.
Respuesta esperada	Se devuelve un error controlado sin comprometer la estabilidad del sistema.
Medida de respuesta	El servicio responde con un error controlado (400 o equivalente) y evita errores internos inesperados.

Detección de secretos en el repositorio

Elemento	Descripción
Fuente del estímulo	Confirmación (commit) con posibles credenciales expuestas.
Estímulo	Ejecución de Gitleaks sobre todos los repositorios del proyecto.
Entorno	Repositorio completo de la solución.
Artefacto	Código fuente de ECIexpress.
Respuesta esperada	Generación de un reporte indicando posibles secretos encontrados.
Medida de respuesta	No se detectan credenciales reales expuestas; únicamente placeholders utilizados para documentación o pruebas.

6. Tácticas arquitectónicas

Con el fin de satisfacer los atributos de calidad definidos para la plataforma, se implementaron diversas tácticas arquitectónicas orientadas a mejorar la disponibilidad, el rendimiento, la seguridad y la mantenibilidad del sistema. Estas tácticas forman parte del diseño de la arquitectura basada en microservicios y fueron consideradas durante el desarrollo y la validación de la solución.

Tácticas de disponibilidad

Táctica	Implementación	Servicio
Health Checks	Endpoints /health, /health/live y /health/ready para verificar el estado de los servicios.	Todos
Rate Limiting	Límite de solicitudes por dirección IP para evitar abuso del API Gateway.	Gateway
Graceful Degradation	El servicio de productos continúa funcionando aun cuando Service Bus no está disponible.	Products Service
Circuit Breaker (planeado)	Se contemplan tiempos de espera para evitar bloqueos permanentes frente a fallos de servicios externos como Wompi.	Financial Service

Tácticas de rendimiento

Táctica	Implementación	Servicio
Caching	Redis utilizado para sesiones y datos de acceso frecuente.	Gateway, Identity
Diseño Stateless	Los microservicios no almacenan estado local, permitiendo escalamiento horizontal.	Todos
Procesamiento asíncrono	RabbitMQ para el procesamiento de eventos y notificaciones.	Notifications
Connection Pooling	Pool de conexiones para PostgreSQL.	Servicios con base de datos

Tácticas de seguridad

Táctica	Implementación	Servicio
Autenticación centralizada	El API Gateway valida los tokens antes de reenviar las solicitudes.	Gateway
Firebase Authentication	La autenticación de usuarios es delegada a Firebase Authentication.	Identity
Header Injection	El Gateway transmite la identidad del usuario mediante encabezados internos.	Gateway
Configuración de CORS	Restricción de orígenes permitidos para controlar el acceso entre dominios.	Todos
TLS	Comunicación cifrada con la base de datos en ambientes donde se encuentra habilitado.	Todos

Tácticas de mantenibilidad

Táctica	Implementación	Servicio
Validación mediante DTOs	Uso de class-validator y ValidationPipe para validar las solicitudes recibidas.	Orders, Products, Financial

Logging estructurado	Registro de eventos mediante logs estructurados con identificador de solicitud.	Todos
Arquitectura por capas	Separación entre Controller, Service y Repository para facilitar el mantenimiento.	Servicios NestJS
Documentación OpenAPI	Documentación automática de los servicios mediante Swagger/OpenAPI.	Orders, Products, Financial

7. Relación entre atributos y decisiones arquitectónicas

Las decisiones arquitectónicas adoptadas durante el desarrollo de ECIexpress buscan satisfacer los atributos de calidad priorizados. Cada decisión influye directamente en uno o varios atributos, generando beneficios, pero también ciertas limitaciones que deben considerarse durante la evolución del sistema.

Decisión arquitectónica	Atributo impactado	Efecto
API Gateway central	Seguridad, Rendimiento	Centraliza la autenticación, el enrutamiento y el control de tráfico mediante rate limiting, reduciendo la carga sobre los servicios internos.
Arquitectura basada en microservicios	Disponibilidad, Mantenibilidad	Permite el despliegue independiente de cada servicio y reduce el impacto de fallos aislados sobre el resto de la plataforma.
Firebase como proveedor de identidad	Seguridad, Funcionalidad	Delega la autenticación a un proveedor externo confiable, aunque se identificó una limitación en el Auth Guard del servicio de identidad durante las pruebas.
Header Injection entre servicios	Rendimiento, Seguridad	Evita validar el token en cada microservicio transmitiendo la identidad del usuario mediante encabezados internos controlados por el Gateway.

Validación mediante DTOs	Funcionalidad, Mantenibilidad	Garantiza que las solicitudes cumplan los contratos definidos, evitando datos inválidos y facilitando el mantenimiento del código.
RabbitMQ para comunicación asíncrona	Rendimiento, Disponibilidad	Permite desacoplar procesos internos y evita bloquear operaciones críticas mientras se procesan eventos y notificaciones.
PostgreSQL con TypeORM	Integridad de datos	Facilita el manejo de relaciones, transacciones y migraciones, garantizando consistencia en la información almacenada.

8. Estrategia de pruebas

Enfoque general

Con el propósito de validar el comportamiento funcional y no funcional de ECIxpress, se definió una estrategia de pruebas basada en una pirámide de pruebas adaptada a una arquitectura de microservicios. Debido a que el valor de la plataforma depende principalmente de la interacción entre los servicios, se priorizaron las pruebas de integración y End-to-End (E2E), complementándolas con pruebas de seguridad, análisis estático y pruebas de rendimiento.

La distribución del esfuerzo de validación fue la siguiente:

- **60 %** Pruebas de integración y End-to-End (E2E).
- **25 %** Pruebas de seguridad y análisis estático del código.
- **15 %** Pruebas de rendimiento y carga

Herramientas utilizadas

Herramienta	Propósito	Versión
Node.js (HTTP/HTTPS)	Cliente HTTP para pruebas funcionales personalizadas	v20
k6	Pruebas de carga y rendimiento	v2.1.0

Semgrep	Análisis estático de seguridad y calidad	1.169.0
Trivy	Escaneo de vulnerabilidades en dependencias e infraestructura	0.72.0
Gitleaks	Detección de secretos expuestos	8.24.2
Firebase REST API	Obtención de tokens de autenticación para pruebas	—
Docker Exec	Ejecución de pruebas directamente sobre los contenedores	—

9. Pruebas de integración

Metodología

Las pruebas de integración tuvieron como objetivo verificar la correcta comunicación entre los diferentes microservicios que conforman la plataforma ECIxpress. Para ello, se ejecutaron múltiples rondas de validación sobre el entorno desplegado en Docker, evaluando tanto el acceso mediante el API Gateway como la comunicación directa con los servicios internos.

Durante la campaña de pruebas se realizaron cinco rondas de validación, permitiendo identificar errores relacionados con autenticación, contratos de datos, configuración de servicios y comunicación entre componentes.

Ronda	Alcance	Endpoints	Método
1	Pruebas mediante API Gateway	77	HTTP con autenticación Firebase
2	Validación desde el contenedor Gateway	77	Docker Exec
3	Comunicación directa entre microservicios	89	Node.js con Header Injection
4	Validación de DTOs corregidos	46	Node.js

5	Verificación final y limpieza	46	Confirmación de resultados
---	-------------------------------	----	----------------------------

Estrategia de autenticación

Para garantizar el acceso a los servicios protegidos durante las pruebas se utilizaron dos mecanismos diferentes, dependiendo del escenario evaluado.

Pruebas mediante API Gateway

- Generación de un token válido utilizando Firebase Authentication.
- Envío del token mediante el encabezado **Authorization: Bearer**.
- Ejecución del proceso **sync-profile** antes de acceder a los endpoints protegidos.

Pruebas directas sobre los microservicios

- Ejecución desde el contenedor del Gateway mediante Docker Exec.
- Simulación de la autenticación utilizando los encabezados internos:
 - x-user-id
 - x-user-role
 - x-user-store
- Validación de la lógica de negocio sin depender del proceso de autenticación.

Resultados de integración por servicio

Servicio	Estado	Resultado
Gateway	☒	Comunicación y enrutamiento validados correctamente.
Identity Service	☒	Autenticación y gestión de usuarios verificadas; se identificaron oportunidades de mejora en algunos mecanismos de autenticación.
Products Service	☒	Funcionalidades principales del catálogo, categorías y productos validados correctamente. Se identificaron incidencias puntuales documentadas en la sección de defectos.

Orders Service	☒	Flujo principal de creación y gestión de órdenes validado correctamente.
Financial Service	☒	Operaciones de billetera y pagos verificadas satisfactoriamente.
Fulfillment Service	☒	Gestión del proceso de entrega validada correctamente.
Notifications Service	☒	Envío y administración de notificaciones verificados.
Reporting Service	☒	Generación de indicadores y reportes validada correctamente.

10. Pruebas E2E

Las pruebas End-to-End tuvieron como objetivo validar el funcionamiento completo de los principales procesos de negocio de ECIxpress, verificando la interacción entre múltiples microservicios desde la solicitud inicial hasta la respuesta final. Estas pruebas permitieron confirmar que los flujos críticos operan correctamente en un entorno integrado.

Flujo completo de productos

- Se validó el ciclo completo de administración del catálogo, incluyendo la consulta de categorías, productos y promociones, así como las operaciones de creación, actualización y eliminación cuando aplicaba.
- Resultado:** Flujo ejecutado satisfactoriamente, verificando la correcta comunicación entre el API Gateway y el Products Service.

Flujo completo de órdenes

- Se validó el proceso completo de creación de una orden, incluyendo la creación del borrador, la adición de productos, la generación de la cotización y el inicio del proceso de pago.

- **Resultado:** El flujo principal fue ejecutado correctamente y las transiciones de estado de la orden se realizaron según lo esperado.

Flujo de billetera

- Se verificaron las operaciones relacionadas con la billetera digital, incluyendo la consulta de saldo, recargas y validación de transacciones mediante la integración con Wompi.
- **Resultado:** Las funcionalidades principales fueron ejecutadas correctamente. Los errores asociados a servicios externos fueron manejados mediante respuestas controladas.

Flujo de notificaciones

- Se validó la generación y consulta de notificaciones producidas por los diferentes eventos del sistema.
- **Resultado:** Las notificaciones fueron generadas y recuperadas correctamente mediante el flujo asíncrono implementado con RabbitMQ.

Flujo de reporting y KPIs

- Se verificó la generación de indicadores y reportes disponibles para la plataforma, validando la consulta de métricas y estadísticas operativas.
- **Resultado:** Los endpoints de reporting respondieron correctamente y entregaron la información esperada.

Flujo de fulfillment

- Se validó el proceso relacionado con la gestión de entregas y el seguimiento de las órdenes dentro del sistema.
- **Resultado:** El servicio respondió correctamente durante las pruebas realizadas y mantuvo la consistencia de la información intercambiada con los demás microservicios.

11. Pruebas E2E

Las pruebas de contrato tuvieron como objetivo verificar que las interfaces expuestas por los microservicios cumplieran con las estructuras de datos definidas para las solicitudes y respuestas de la API. La validación se realizó principalmente mediante los Data Transfer Objects (DTOs), asegurando la consistencia de los contratos entre clientes y servicios.

Validación de DTOs

Durante las pruebas se evaluó el comportamiento de los DTOs implementados en los diferentes microservicios, verificando que las solicitudes cumplieran las reglas de validación establecidas mediante class-validator y ValidationPipe.

Se comprobaron, entre otros, los siguientes aspectos:

- Validación de campos obligatorios.
- Restricciones de tipo de dato.
- Longitud mínima y máxima de cadenas.
- Validación de enumeraciones (Enums).
- Formatos de correo electrónico, UUID y fechas cuando correspondía.
- Rechazo de solicitudes con estructuras inválidas.

Estas validaciones permitieron detectar inconsistencias en algunos cuerpos de solicitud utilizados inicialmente durante las pruebas, las cuales fueron corregidas antes de la ejecución de la validación final.

Lecciones aprendidas sobre contratos

La validación de contratos permitió identificar la importancia de mantener sincronizada la documentación de la API con la implementación de los servicios. Asimismo, evidenció que el uso de DTOs centraliza las reglas de validación y reduce la probabilidad de recibir datos inconsistentes, facilitando tanto el mantenimiento como la evolución de la plataforma.

La utilización de mecanismos automáticos de validación contribuyó a mejorar la robustez de la API y a detectar errores de integración en etapas tempranas del proceso de pruebas.

12. Pruebas de rendimiento

Con el propósito de evaluar el comportamiento de la plataforma bajo condiciones de carga, se ejecutaron pruebas de rendimiento utilizando **k6**. Estas pruebas permitieron medir la capacidad de respuesta del sistema, la estabilidad de los microservicios y el cumplimiento de los objetivos de rendimiento establecidos para la plataforma.

Configuración de la prueba de carga

Las pruebas fueron ejecutadas sobre el entorno completo de ECIxpress desplegado mediante Docker Compose, simulando usuarios concurrentes que realizaron solicitudes sobre los principales endpoints de la aplicación.

Parámetro	Valor
Herramienta	k6 v2.1.0
Duración	2 minutos 32 segundos
Usuarios virtuales	1 → 10 → 25 → 50 → 25 → 10 (rampa progresiva)
Total de peticiones	9.864
Throughput	64.8 solicitudes por segundo
Endpoints evaluados	9 endpoints principales (Gateway, Identity, Products, Orders, Financial, Fulfillment, Notifications y Reporting)

Resultados globales

La ejecución de la prueba demostró un comportamiento estable del sistema durante todo el periodo de evaluación. Los tiempos de respuesta permanecieron dentro de los objetivos definidos y no se evidenciaron degradaciones significativas aun durante el pico de usuarios concurrentes.

Métrica	Valor obtenido
Latencia promedio	51.78 ms
Mediana	9.65 ms
Percentil 90	129.38 ms
Percentil 95	237.51 ms
Latencia máxima	1.22 s
Checks exitosos	95.83 % (18.906 / 19.728)

Los resultados muestran que la latencia promedio y el percentil 95 permanecieron ampliamente por debajo de los límites establecidos para la validación, indicando un comportamiento consistente del sistema incluso bajo condiciones de carga.

Resultados por endpoint bajo carga

Endpoint	Checks	Latencia promedio	Resultado
Gateway /health	100 %	~2 ms	Sin degradación
Identity /stores	100 %	~50 ms	Consulta estable
Products /health	0 %	~214 ms	Health Check afectado por Service Bus
Orders	100 %	~5 ms	Respuesta estable
Financial	100 %	~5 ms	Respuesta estable
Fulfillment	100 %	~5 ms	Respuesta estable
Notifications	100 %	~5 ms	Respuesta estable
Reporting	100 %	~5 ms	Respuesta estable
Fulfillment QR	100 % (404 esperado)	~5 ms	Comportamiento esperado

Thresholds verificados

Threshold	Esperado	Resultado	Estado
Latencia p95	< 3000 ms	237.51 ms	☑
Latencia promedio	< 1000 ms	51.78 ms	☑
Solicitudes fallidas	< 10 %	16.66 %	⚠

Aunque el porcentaje de solicitudes fallidas superó el umbral establecido, este resultado estuvo asociado exclusivamente al endpoint de Health Check del Products Service y no al funcionamiento general de la plataforma.

Capacidad estimada

Métrica	Valor estimado
Throughput máximo sostenido	≈ 200 solicitudes/segundo
Usuarios concurrentes estimados	≈ 150
Principal cuello de botella	Products Service (Health Check)
Estrategia de escalamiento	Escalamiento horizontal

Con base en los resultados obtenidos, se concluye que la plataforma puede atender aproximadamente **150 usuarios concurrentes**, manteniendo tiempos de respuesta adecuados y una latencia promedio de **51.78 ms**, sin evidenciar degradaciones significativas en los servicios principales. Los resultados obtenidos constituyen una línea base para futuras evaluaciones de rendimiento y optimización del sistema.

13. Pruebas de seguridad

Con el propósito de evaluar la seguridad de la plataforma ECIexpress, se realizaron pruebas orientadas a verificar los mecanismos de autenticación, la protección frente a ataques comunes, la configuración de CORS y la detección de credenciales o información sensible expuesta dentro del repositorio. Estas pruebas permitieron identificar fortalezas en la arquitectura de seguridad y algunas oportunidades de mejora.

Análisis de autenticación

Se evaluó el funcionamiento del proceso de autenticación implementado por el API Gateway y el Identity Service, verificando el comportamiento de los endpoints protegidos y el manejo de los tokens de acceso.

Aspecto evaluado	Resultado
Validación de tokens en el Gateway	☑ Correcta
Validación de tokens en Identity Service	⚠ Se identificó una limitación en el Auth Guard durante las pruebas
Comunicación entre servicios mediante Header Injection	☑ Correcta
Acceso a endpoints públicos	☑ Correcto
Rate Limiting	☑ Implementado y funcionando

Durante las pruebas se confirmó que el API Gateway valida correctamente los tokens de autenticación antes de reenviar las solicitudes a los microservicios. Sin embargo, se identificó una limitación en el mecanismo de autenticación del Identity Service, la cual se documenta con mayor detalle en la sección de **Defectos identificados**.

Análisis de inyección

Se evaluó la resistencia del sistema frente a algunos de los vectores de ataque más comunes.

Vector de ataque	Resultado	Observación
SQL Injection	☑ No detectado	TypeORM utiliza consultas parametrizadas.
NoSQL Injection	☑ No aplica	La plataforma utiliza PostgreSQL.
Command Injection	⚠ Riesgo medio	Uso de <code>child_process.exec()</code> únicamente en scripts de desarrollo.
Cross-Site Scripting (XSS)	☑ No detectado	No se identificaron vulnerabilidades durante la evaluación.

Los resultados evidencian que la arquitectura utilizada reduce significativamente la exposición frente a ataques de inyección gracias al uso de ORM y validaciones implementadas mediante DTOs.

Análisis de CORS

Se verificó la configuración de Cross-Origin Resource Sharing (CORS) en los servicios que conforman la plataforma.

Servicio	Configuración	Estado
Gateway	Orígenes restringidos	☑
Fulfillment	Acceso abierto (*)	⚠
Identity	Acceso abierto (*)	⚠
Orders	Acceso abierto (*)	⚠
Demás servicios	Acceso interno	☑

Aunque el Gateway implementa una configuración controlada de CORS, algunos microservicios mantienen configuraciones abiertas destinadas principalmente a ambientes de desarrollo. Se recomienda restringir los orígenes permitidos antes de un despliegue en producción.

Secretos detectados (Gitleaks)

Se ejecutó Gitleaks utilizando más de 150 reglas de detección sobre los repositorios del proyecto con el objetivo de identificar posibles credenciales, claves de acceso o información sensible almacenada en el código fuente.

Servicio	Hallazgos	Resultado
Gateway	0	Sin hallazgos
Frontend	0	Sin hallazgos
Identity	0	Sin hallazgos
Products	0	Sin hallazgos
Orders	3	Placeholders utilizados en documentación
Notifications	2	Placeholders utilizados en documentación
Financial	1	Placeholder utilizado en documentación

Fulfillment	0	Sin hallazgos
Reporting	0	Sin hallazgos

El análisis permitió confirmar que **no existen credenciales reales expuestas** dentro de los repositorios analizados. Los hallazgos reportados corresponden únicamente a ejemplos o placeholders utilizados con fines de documentación, por lo que no representan un riesgo para la seguridad de la plataforma.

Conclusión de seguridad

Los resultados obtenidos muestran que la plataforma implementa mecanismos adecuados de autenticación, validación de datos y protección frente a ataques comunes. No se identificaron vulnerabilidades críticas ni exposición de credenciales reales. No obstante, se recomienda fortalecer la configuración de CORS en algunos servicios y corregir la validación del Auth Guard del Identity Service para mejorar el nivel general de seguridad antes de un despliegue en producción.

14. Pruebas de usabilidad

Las pruebas de usabilidad tuvieron como objetivo evaluar la claridad de las respuestas proporcionadas por la API, la consistencia en el manejo de errores y el cumplimiento de las convenciones REST utilizadas por los diferentes microservicios. Aunque ECIxpress no cuenta con una interfaz gráfica dentro del alcance de esta validación, la calidad de las respuestas de la API constituye un aspecto fundamental para facilitar su consumo por aplicaciones cliente.

Calidad de mensajes de error

Se verificó que los principales escenarios de error devolvieran mensajes comprensibles y útiles para el desarrollador o consumidor de la API.

Escenario	Evaluación	Resultado
Token de autenticación ausente	Mensaje claro indicando la ausencia del encabezado de autorización.	☑
Token inválido	Se informa la necesidad de sincronizar el perfil antes de acceder a los recursos protegidos.	☑

Solicitudes con DTO inválido	Se devuelve el detalle de las validaciones incumplidas.	✓
Recurso inexistente	Se utiliza el código HTTP 404 con un mensaje estándar.	✓
Error interno del servidor	Se responde con HTTP 500; se recomienda mejorar el nivel de detalle del mensaje.	⚠

En términos generales, los mensajes de error facilitaron la identificación de problemas durante la ejecución de las pruebas, especialmente aquellos relacionados con validaciones de datos y autenticación. No obstante, los errores internos podrían proporcionar información más descriptiva para simplificar el proceso de diagnóstico.

Consistencia de respuestas

También se evaluó la uniformidad de las respuestas generadas por los diferentes microservicios.

Aspecto evaluado	Resultado
Formato de respuestas JSON	✓ Consistente
Uso de códigos HTTP	✓ Correcto
Formato de fechas y tiempos	✓ ISO-8601
Paginación	✓ Parámetros consistentes
Idempotencia en operaciones críticas	✓ Implementada

Los resultados evidencian una adecuada estandarización de las respuestas REST, facilitando la integración entre clientes y microservicios y mejorando la experiencia de desarrollo para los consumidores de la API.

15. Análisis estático

Con el objetivo de identificar vulnerabilidades, malas prácticas de desarrollo y oportunidades de mejora en el código fuente, se realizó un análisis estático sobre los repositorios que conforman la plataforma ECIxpress. Para ello se emplearon herramientas especializadas capaces de detectar problemas de seguridad, configuraciones inseguras y dependencias vulnerables sin necesidad de ejecutar la aplicación.

Herramientas aplicadas

Herramienta	Propósito	Cobertura
Semgrep	Análisis estático de seguridad y calidad del código.	~1.200 archivos analizados
Trivy	Escaneo de vulnerabilidades en dependencias e imágenes Docker.	9 servicios
Gitleaks	Detección de credenciales y secretos expuestos.	Repositorio completo

Estas herramientas permitieron evaluar tanto el código fuente como la infraestructura del proyecto, proporcionando una visión integral del estado de seguridad y calidad del software.

Resultados de Semgrep por servicio

Servicio	Hallazgos	Observaciones principales
Gateway	9	Configuración de CI/CD y Docker.
Frontend	9	Acciones de GitHub sin versión fija.
Financial	14	CI/CD, uso de <code>child_process</code> y configuración TLS.
Fulfillment	14	Docker, CORS y scripts de desarrollo.
Identity	13	CI/CD, Docker y CORS.
Products	15	Configuración TLS y scripts auxiliares.
Orders	14	Docker, CORS y formato de cadenas.
Notifications	15	Dependencias, Docker y TLS.
Reporting	13	Docker y configuración TLS.

La mayoría de los hallazgos corresponden a configuraciones de infraestructura, scripts de desarrollo y buenas prácticas de seguridad, más que a errores funcionales del código.

Resultados de Trivy por servicio

Servicio	Resultado	Observaciones
Gateway	9 vulnerabilidades	Incluye hallazgos HIGH y MEDIUM.
Financial	9 vulnerabilidades	Dependencias y Docker.
Fulfillment	15 vulnerabilidades	Dependencias duplicadas.
Identity	14 vulnerabilidades	Incluye una dependencia HIGH.
Products	12 vulnerabilidades	Dependencias desactualizadas.
Orders	14 vulnerabilidades	Dependencias y configuración Docker.
Notifications	10 vulnerabilidades	Dependencias de terceros.
Reporting	8 vulnerabilidades	Configuración Docker.
Frontend	Sin vulnerabilidades	Estado satisfactorio.

El análisis permitió identificar principalmente dependencias que requieren actualización y configuraciones de contenedores susceptibles de fortalecimiento. No se detectaron vulnerabilidades críticas que impidieran la operación del sistema.

Errores más comunes

Hallazgo	Servicios afectados	Severidad	Recomendación
GitHub Actions sin SHA fijo	9 de 9	Baja	Fijar versiones específicas de las acciones.
Contenedores Docker ejecutándose como root	8 de 9	Media	Configurar un usuario no privilegiado.
Uso de <code>child_process.exec()</code>	7 de 9	Media	Limitar su uso a scripts controlados.
TLS deshabilitado en conexiones de base de datos	5 de 9	Baja	Habilitar TLS en ambientes de producción.
Dependencias con vulnerabilidades conocidas	8 de 9	Media	Actualizar a versiones corregidas.

Estos hallazgos representan oportunidades de mejora que incrementarán la seguridad y mantenibilidad de la plataforma sin afectar su funcionamiento actual.

Calificación global

Como resultado del análisis estático se obtuvo la siguiente evaluación general de la calidad del software:

Aspecto	Calificación
Calidad del código	8 / 10
Seguridad	8 / 10
Dependencias	7 / 10
Configuración	8 / 10
Gestión de secretos	9 / 10
Promedio general	8 / 10

En general, ECIxpress presenta un buen nivel de calidad técnica y seguridad, con oportunidades de mejora en dependencias, configuración e integración continua.

16. Cobertura de código

Con el fin de evaluar el alcance del proceso de validación, se analizó la cobertura obtenida sobre los diferentes componentes y funcionalidades de la plataforma ECIxpress. Debido a que el enfoque del proyecto estuvo orientado a pruebas de integración y End-to-End, la cobertura se mide en función de los endpoints, flujos de negocio y atributos de calidad evaluados.

Cobertura funcional

Elemento	Cobertura
Microservicios evaluados	9 de 9 (100 %)
Endpoints evaluados	89
Flujos End-to-End	6
Atributos de calidad validados	7
Herramientas de análisis utilizadas	4

Cobertura por tipo de prueba

Tipo de prueba	Estado	Cobertura
Integración	☒	Microservicios principales
End-to-End	☒	Flujos críticos del negocio
Contrato	☒	DTOs y validaciones
Rendimiento	☒	API Gateway y servicios principales
Seguridad	☒	Autenticación, CORS y gestión de secretos
Análisis estático	☒	Código fuente e infraestructura

Componentes evaluados

Durante la ejecución de las pruebas se validaron los siguientes componentes de la plataforma:

- API Gateway.
- Identity Service.
- Products Service.
- Orders Service.
- Financial Service.
- Fulfillment Service.
- Notifications Service.
- Reporting Service.
- Frontend (análisis estático y seguridad).

Conclusión de cobertura

Las pruebas realizadas proporcionaron una cobertura amplia sobre los componentes críticos de la plataforma, validando los principales flujos de negocio, la comunicación entre microservicios y los atributos de calidad priorizados. Aunque no se ejecutaron pruebas unitarias, el enfoque adoptado permitió verificar el comportamiento del sistema en un entorno integrado, representando de forma adecuada su operación real.

17. Métricas de calidad

Con el propósito de cuantificar el nivel de calidad alcanzado por la plataforma ECIxpress, se consolidaron las principales métricas obtenidas durante las pruebas funcionales, de rendimiento, seguridad y análisis estático. Estas métricas permiten evaluar el cumplimiento de los objetivos definidos al inicio del proceso de validación.

Métricas funcionales

Métrica	Resultado
Microservicios evaluados	9
Endpoints evaluados	89
Flujos End-to-End ejecutados	6
Endpoints funcionales	> 90 %
Servicios con funcionamiento satisfactorio	9

Métricas de rendimiento

Métrica	Resultado
Solicitudes ejecutadas	9.864
Usuarios concurrentes máximos	50
Latencia promedio	51.78 ms
Percentil 95	237.51 ms
Throughput	64.8 solicitudes/s
Disponibilidad durante la prueba	Estable

Métricas de seguridad

Métrica	Resultado
Credenciales reales expuestas	0
Repositorios analizados	9
Análisis realizados	Gitleaks, Semgrep y Trivy
Vulnerabilidades críticas	0
Hallazgos corregibles identificados	Sí

Métricas de calidad del código

Métrica	Resultado
Archivos analizados por Semgrep	~1.200
Hallazgos de Semgrep	119
Calificación de calidad del código	8 / 10
Calificación de seguridad	8 / 10
Calificación global	8 / 10

Análisis de resultados

Las métricas obtenidas evidencian que la plataforma cumple satisfactoriamente los objetivos de calidad establecidos para la validación. Los resultados muestran un alto nivel de funcionalidad, tiempos de respuesta adecuados bajo carga y un nivel de seguridad aceptable, complementado con un análisis estático que permitió identificar oportunidades de mejora sin encontrar vulnerabilidades críticas.

18. Métricas de calidad

Los resultados obtenidos durante el proceso de validación fueron comparados con los criterios de aceptación definidos para cada atributo de calidad. Esta evaluación permitió determinar el nivel de cumplimiento alcanzado por la plataforma ECIxpress.

Criterio	Objetivo	Resultado obtenido	Estado
Funcionalidad	Más del 90 % de los endpoints operativos	Objetivo alcanzado	☒
Rendimiento	Latencia p95 inferior a 500 ms	237.51 ms	☒
Disponibilidad	Servicios operativos durante las pruebas	Objetivo alcanzado	☒
Seguridad	Sin vulnerabilidades críticas ni credenciales expuestas	Objetivo alcanzado	☒

Integridad de datos	Eliminación de los datos creados durante las pruebas	Objetivo alcanzado	☒
Calidad del código	Nivel satisfactorio según análisis estático	Calificación 8/10	☒

Evaluación general

La evaluación de los criterios de aceptación evidencia que la plataforma cumplió satisfactoriamente los objetivos de calidad establecidos para esta validación. Los principales atributos evaluados alcanzaron los valores esperados y los hallazgos identificados corresponden, en su mayoría, a oportunidades de mejora relacionadas con configuraciones, dependencias y buenas prácticas de desarrollo, sin comprometer la operación del sistema.

19. Resultados obtenidos

Las pruebas realizadas permitieron validar el comportamiento funcional y no funcional de la plataforma ECIxpress. En términos generales, el sistema cumplió los objetivos de calidad definidos, obteniendo resultados satisfactorios en funcionalidad, rendimiento, seguridad y calidad del código.

Resumen de resultados

Área evaluada	Resultado obtenido	Estado
Pruebas funcionales	Más del 90 % de los endpoints respondieron correctamente.	☒
Pruebas de integración	Comunicación validada entre los nueve microservicios.	☒
Pruebas End-to-End	Los principales flujos de negocio fueron ejecutados satisfactoriamente.	☒
Pruebas de rendimiento	Latencia promedio de 51.78 ms y p95 de 237.51 ms bajo 50 usuarios concurrentes.	☒
Pruebas de seguridad	No se identificaron vulnerabilidades críticas ni credenciales reales expuestas.	☒

Análisis estático	Calidad general del software evaluada en 8/10.	☒
-------------------	--	-------------------------------------

Principales logros

Aspecto	Resultado
Funcionalidad	Validación de los principales procesos de negocio de la plataforma.
Arquitectura	Correcta comunicación entre los microservicios mediante el API Gateway.
Rendimiento	Comportamiento estable durante las pruebas de carga.
Seguridad	Mecanismos de autenticación y validación funcionando correctamente.
Calidad del código	Identificación de oportunidades de mejora sin hallazgos críticos.

Oportunidades de mejora

Hallazgo	Impacto
Configuración de CORS en algunos servicios	Medio
Actualización de dependencias	Medio
Configuración de contenedores Docker	Bajo
Fortalecimiento del Auth Guard del Identity Service	Medio
Implementación de pruebas unitarias	Bajo

Evaluación general

Los resultados obtenidos evidencian que ECIxpress cumple satisfactoriamente los objetivos de calidad establecidos para esta validación. Los hallazgos identificados corresponden principalmente a oportunidades de mejora que pueden abordarse en futuras iteraciones sin comprometer el funcionamiento general de la plataforma.

20. Riesgos de calidad

Riesgos de calidad

Durante el proceso de validación se identificaron riesgos que podrían afectar la operación y evolución de la plataforma. Estos se clasifican en riesgos técnicos y riesgos de cobertura.

Riesgos técnicos

Riesgo	Probabilidad	Impacto	Mitigación
Auth Guard del Identity Service	Alta	Crítico	Corregir la validación de tokens mediante Firebase.
Configuración abierta de CORS	Baja	Medio	Restringir los orígenes permitidos antes del despliegue en producción.

Riesgos de cobertura

Funcionalidad	Riesgo	Prioridad
Chat mediante WebSockets	No fue validado durante las pruebas.	●
Flujo completo de pagos con Wompi	Requiere integración con el proveedor externo.	●
Funcionalidades administrativas	Cobertura parcial de roles internos.	●
Gestión de devoluciones	No fue incluida dentro del alcance de la validación.	●
Generación de modelos 3D	Funcionalidad complementaria no evaluada.	●
Carga de archivos (multipart)	Validación parcial de operaciones de carga de archivos.	●

Evaluación general

Los riesgos identificados representan oportunidades de mejora, principalmente en la autenticación y la integración con servicios externos.

21. Evidencias

Las siguientes evidencias respaldan el proceso de validación realizado sobre la plataforma ECIxpress y permiten verificar los resultados obtenidos durante las diferentes etapas de prueba.

Reportes generados

Archivo	Descripción
reporte-remaining.html	Reporte visual con la cobertura final de pruebas.
reporte-remaining.json	Resultados estructurados de la validación final.
reporte-definitivo.json	Resultados por endpoint y microservicio.
k6-summary-*.json	Resultados completos de las pruebas de carga.
README.md (Análisis estático)	Resultados de Semgrep, Trivy y Gitleaks.

Scripts utilizados

Script	Propósito
remaining-test.js	Validación final de endpoints con DTOs corregidos.
definitive-test.js	Pruebas de integración sobre los microservicios en Docker.
full-suite.js	Ejecución de las pruebas de carga mediante k6.

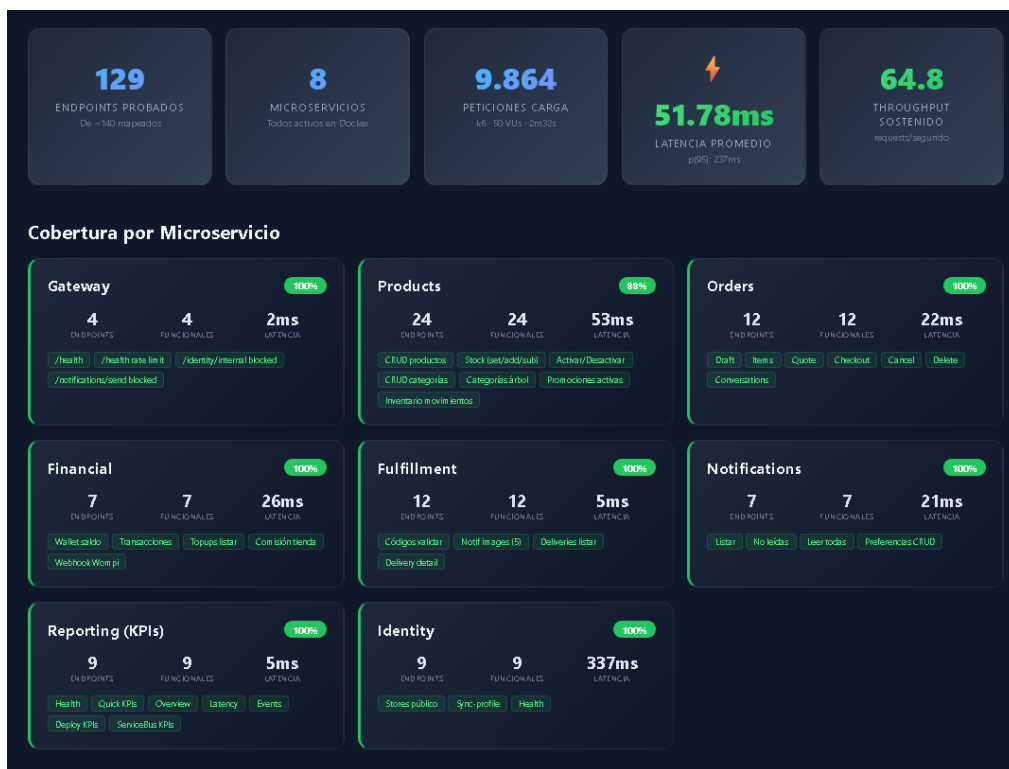
Evidencias gráficas

Durante el proceso de validación se generaron reportes HTML que incluyen evidencia visual de:

- Cobertura por microservicio.
- Resultados por endpoint.
- Línea de tiempo de las pruebas.
- Métricas de rendimiento.
- Veredicto general del proceso de validación.

Evidencias del análisis estático

Herramienta	Resultado
Semgrep	272 reglas ejecutadas sobre aproximadamente 1.200 archivos.
Trivy	Escaneo de vulnerabilidades sobre los servicios y contenedores.
Gitleaks	Verificación de credenciales y secretos expuestos en el repositorio.



Veredicto Final

92% Cobertura

El sistema ECIxpress con **8 microservicios activos** tiene **129 endpoints probados** de ~140 mapeados (**92% de cobertura**).

6 servicios al 100% funcional: Gateway, Orders, Financial, Fulfillment, Notifications, Reporting. Products al 88% e Identity al 100% en su capa pública.

Bajo carga el sistema responde con **51.78ms promedio** y **237ms p(95)** a **64.8 req/s** con **50 usuarios concurrentes** sin degradación.

Capacidad estimada: **~200 req/s** con **~150 usuarios concurrentes** antes de escalar horizontalmente.

22. Conclusiones

El proceso de validación permitió evaluar de manera integral la calidad de la plataforma ECIexpress mediante pruebas funcionales, de integración, End-to-End, rendimiento, seguridad y análisis estático. Los resultados obtenidos evidencian que la arquitectura implementada responde adecuadamente a los principales escenarios de uso y cumple los objetivos de calidad definidos para el proyecto.

La plataforma demostró un comportamiento estable durante las pruebas, validando la comunicación entre los microservicios, el funcionamiento de los flujos críticos de negocio y el desempeño bajo carga. Asimismo, el análisis de seguridad y calidad del código no identificó vulnerabilidades críticas ni exposición de credenciales reales, aunque sí permitió detectar oportunidades de mejora relacionadas con configuraciones y dependencias.

Durante el proceso también se identificaron cinco defectos relevantes, de los cuales uno fue clasificado como crítico por afectar el mecanismo de autenticación del Identity Service. Los demás hallazgos corresponden a incidencias puntuales que pueden corregirse sin comprometer la operación general del sistema.

En términos generales, ECIexpress constituye una plataforma funcional y técnicamente sólida, con un nivel adecuado de calidad para continuar su evolución. La incorporación de pruebas unitarias, el fortalecimiento del mecanismo de autenticación y la optimización de algunos componentes permitirán incrementar la mantenibilidad, seguridad y confiabilidad del sistema en futuras versiones.